

AI Service Desk

Technical Documentation

Date: July 24, 2026, **Version:** 1.0

Revision History

Version	Date	Author	Description
1.0	24-Jul-2026	Sivakarthick B	Initial technical documentation according to project source code

Table Contents

Technical Documentation.....	1
1. Introduction.....	1
1.1 Purpose.....	1
1.2 Scope.....	1
1.3 Intended Audience.....	1
1.4 References.....	1
1.5 Acronyms and Definitions.....	1
2. System Overview.....	2
2.1 High-Level Architecture.....	2
2.2 System flow - Key Components.....	4
2.3 User Roles and Access Levels.....	4
3. Functional Requirements.....	4
3.1 Module/Feature Name.....	4
3.2 Inputs/Outputs.....	7
3.3 Business Rules.....	7
3.4 Error Handling.....	7
4. Non-Functional Requirements.....	8
4.1 Performance.....	8
4.2 Security.....	8
4.3 Availability.....	8
4.4 Maintainability.....	8
5. Data Model / Database Design.....	9
5.1 Entity-Relationship Diagram.....	9
5.2 Tables and Fields.....	9
5.3 Indexes and Keys.....	9
5.4 Sample Data.....	9
6. Integration Points.....	10
6.1 External Systems.....	10
6.2 APIs Used.....	10
6.3 Data Flow Diagram.....	10
6.4 Authentication/Authorization.....	10
7. Test Strategy.....	10

7.1 Test Scenarios.....	10
7.2 Expected Results.....	11
7.3 Traceability to Requirements.....	11
8. Deployment & Configuration.....	12
8.1 Environments.....	12
8.2 Setup Steps.....	12
8.3 Configuration Files and Parameters.....	12
9. Appendix.....	13
Glossary.....	13
Screenshots.....	14
Code Snippets.....	15
References.....	15

1. Introduction

1.1 Purpose

This document describes the technical implementation of the AI Service Desk backend application, an asynchronous REST API for creating, tracking, and resolving support tickets, with an AI-assisted ticket summarization feature. It is written based on the source code (FastAPI application, SQLAlchemy models, Alembic migrations, Docker configuration, and automated test suite) and aims to give developers, reviewers, and operators an accurate reference for how the system is built and how it runs.

1.2 Scope

This document covers the backend service according to the source code: the FastAPI application (app/), the ticket data model and database access layer, the AI summarization integration with Amazon Bedrock, Alembic database migrations, the automated test suite, and the Docker-based packaging of the service. No frontend/client application, infrastructure-as-code, or CI/CD pipeline definitions were present in the supplied source code as of now, so these are out of scope and are marked accordingly where relevant.

1.3 Intended Audience

- Backend developers extending or maintaining the ticketing API
- QA engineers designing or reviewing automated test coverage
- DevOps/engineers responsible for building, deploying, and running the Docker image
- Technical reviewers or new team members onboarding onto the project

1.4 References

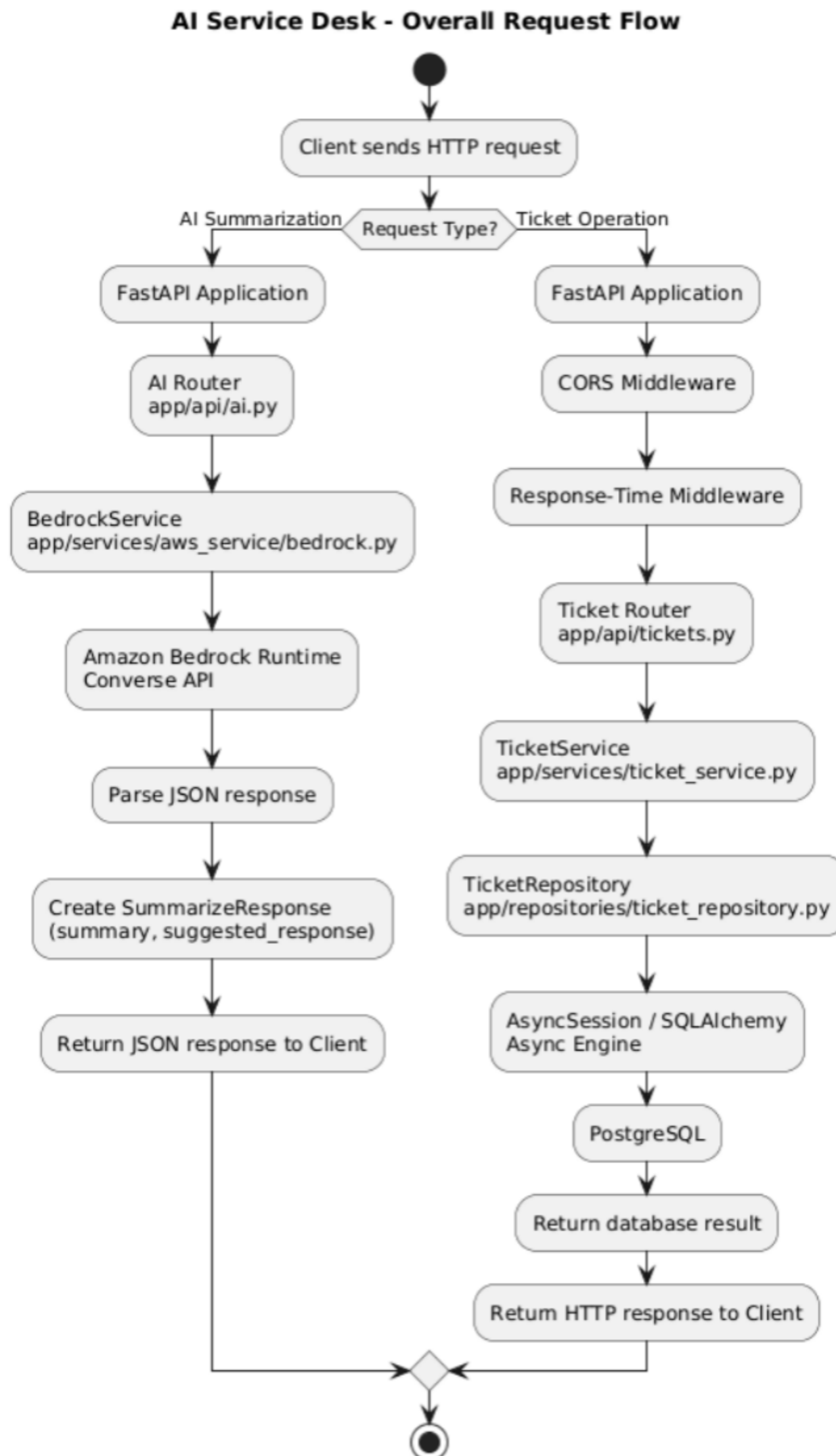
- FastAPI documentation – <https://fastapi.tiangolo.com/>
- SQLAlchemy 2.0 (asyncio) documentation – <https://docs.sqlalchemy.org/>
- Alembic documentation – <https://alembic.sqlalchemy.org/>
- AWS Bedrock Runtime API documentation -<https://docs.aws.amazon.com/bedrock>

1.5 Acronyms and Definitions

Term	Definition
API	Application Programming Interface
REST	Representational State Transfer
ORM	Object-Relational Mapper (SQLAlchemy)
ASGI	Asynchronous Server Gateway Interface (used by Uvicorn/FastAPI)
UUID	Universally Unique Identifier – used as the ticket primary key
CRUD	Create, Read, Update, Delete
CORS	Cross-Origin Resource Sharing
DTO/Schema	Data Transfer Object – implemented via Pydantic models in app/schemas
LLM	Large Language Model, accessed through Amazon Bedrock

2. System Overview

2.1 High-Level Architecture



2.1.1 Technology Stack

Layer	Technology
Language / runtime	Python (requires-python >=3.12; Dockerfile uses python:3.12-slim)
Web framework	FastAPI 0.139.2 running on Uvicorn (ASGI) 0.51.0
ORM / DB access	SQLAlchemy 2.0.51 (async) with the asyncpg 0.31.0 driver
Database	PostgreSQL (via DATABASE_URL, e.g. postgresql+asyncpg://...)
Schema migrations	Alembic 1.18.5
Data validation / config	Pydantic 2.13.4 and pydantic-settings 2.14.2
AI integration	boto3 1.43.54 (Amazon Bedrock Runtime – Converse API)
Testing	pytest 9.1.1, pytest-asyncio 1.4.0, httpx 0.28.1 (ASGI test client)
Load / performance testing	locust 2.46.1 (locust.py), cProfile (profile_test.py)
Containerization	Docker (python:3.12-slim base image)

2.1.2 Project Directory Structure

Path	Purpose
app/main.py	FastAPI application factory: lifespan handler, CORS and response-time middleware, /health and /ready endpoints, router registration
app/api/tickets.py	Ticket router – HTTP endpoints for ticket CRUD operations
app/api/ai.py	AI router – POST /ai/summarize endpoint
app/core/config.py	Pydantic-settings Settings class loading configuration from environment/.env
app/core/deps.py	FastAPI dependency get_db() providing a request-scoped AsyncSession with commit/rollback handling
app/db/database.py	Async SQLAlchemy engine creation and session-maker factory
app/db/base.py	Declarative Base class used by ORM models
app/models/ticket.py	Ticket ORM model and the Priority/Status enums
app/schemas/ticket.py	Pydantic request/response schemas (TicketCreate, TicketUpdate, TicketResponse, SummarizeRequest, SummarizeResponse)
app/repositories/ticket_repository.py	Data-access layer – raw SQLAlchemy queries against the tickets table
app/services/ticket_service.py	Business logic for ticket CRUD, including the closed-ticket update rule
app/services/aws_service/bedrock.py	BedrockService/FakeBedrockService wrappers around the Amazon Bedrock Converse API
app/services/aws_service/prompt_templates.py	Prompt template(s) used to build the Bedrock summarization prompt
alembic/	Alembic migration environment (env.py) and versioned migration scripts
tests/unit/	Unit tests for Pydantic schema validation (TicketCreate, TicketUpdate)
tests/integration/	Integration tests exercising the API end-to-end via an ASGI test client and a real test database
tests/conftest.py	Pytest fixtures: test database setup/teardown, per-test DB session and HTTP client with dependency override
locust.py	Locust load-testing scenario for /health, GET /tickets, and POST /tickets
profile_test.py	Standalone cProfile-based performance profiling script for the ticket service CRUD flow

Dockerfile	Container image definition for the FastAPI service
alembic.ini	Alembic configuration (migration script location, logging)
pyproject.toml / requirements.txt / uv.lock	Python dependency declarations

AI Service Desk is a layered, asynchronous FastAPI application. Incoming HTTP requests pass through CORS and response-timing middleware, are routed to a router module (Tickets or AI), delegated to a service class containing business logic, and persisted through a repository class that talks to PostgreSQL using SQLAlchemy's async engine/session. The AI router instead delegates to a Bedrock service wrapper that calls Amazon Bedrock's Converse API.

2.2 System flow - Key Components

AI summarization flow: Client → FastAPI app → AI router (app/api/ai.py) → BedrockService (app/services/aws_service/bedrock.py) → Amazon Bedrock Runtime Converse API → JSON response parsed into a SummarizeResponse (summary and suggested_response). Bedrock or parsing failures are translated into a BedrockServiceError and surfaced to the client as an HTTP 502 response.

Ticket request flow: Client → FastAPI app (CORS middleware → response-time middleware) → Ticket router (app/api/tickets.py) → TicketService (app/services/ticket_service.py) → TicketRepository (app/repositories/ticket_repository.py) → AsyncSession / SQLAlchemy async engine → PostgreSQL. The database session is supplied per request by the get_db() dependency, which commits on success and rolls back on exception.

2.3 User Roles and Access Levels

No user accounts, roles, authentication, or role-based access control were not there all endpoints are currently unauthenticated.

3. Functional Requirements

3.1 Module/Feature Name

3.1.1 Ticket Management Module (app/api/tickets.py)

Router prefix: /tickets. Implements create, list, retrieve, update, and delete operations for support tickets, delegating business rules to TicketService.

Endpoint	Method	Purpose	Success Code
/tickets/	POST	Create a new ticket	201 Created
/tickets/	GET	List all tickets, newest first (ordered by created_at desc)	200 OK
/tickets/{ticket_id}	GET	Retrieve a single ticket by UUID	200 OK
/tickets/{ticket_id}	PUT	Update a ticket's	200 OK

		title/description/priority/status	
/tickets/{ticket_id}	DELETE	Delete a ticket by UUID	200 OK

POST /tickets/ – Create Ticket

Request body (TicketCreate): title (string, 1–200 chars, whitespace-stripped, required), description (string, 1–5000 chars, whitespace-stripped, required), priority (High | Medium | Low, default Medium), assignee_email (string, optional, not format-validated).

```
{
  "title": "Login failed",
  "description": "Employee login was declined",
  "priority": "High",
  "assignee_email": "support@example.com"
}
```

Response (201, TicketResponse): id (UUID), title, description, priority, assignee_email, status (defaults to Open), created_at, updated_at.

```
{
  "id": "3f9a9f0e-2c3b-4a11-9b34-9d2e1a6d7c11",
  "title": "Login failed",
  "description": "Employee login was declined",
  "priority": "High",
  "assignee_email": "support@example.com",
  "status": "Open",
  "created_at": "2026-07-24T09:00:00+00:00",
  "updated_at": "2026-07-24T09:00:00+00:00"
}
```

Error responses: 422 Unprocessable Entity for schema validation failures (missing/blank title or description, title/description exceeding length limits, invalid priority value).

GET /tickets/ – List Tickets

No parameters. Returns a JSON array of TicketResponse objects ordered by created_at descending. Success: 200 OK.

GET /tickets/{ticket_id} – Get Ticket

Path parameter: ticket_id (UUID). Success: 200 OK with a TicketResponse body. Error responses: 404 Not Found ("Ticket not found") when no ticket with that id exists; 422 Unprocessable Entity when ticket_id is not a valid UUID.

PUT /tickets/{ticket_id} – Update Ticket

Path parameter: ticket_id (UUID). Request body (TicketUpdate): title, description, priority, and status are all optional – only supplied fields are updated (partial update via model_dump(exclude_unset=True)).

```
{
  "title": "Updated title",
}
```

```
"priority": "High"
}
```

Success: 200 OK with the updated TicketResponse. Error responses: 404 Not Found ("Ticket not found"); 400 Bad Request ("Closed tickets cannot be updated") if the ticket's status is already Closed; 422 Unprocessable Entity for invalid field values.

DELETE /tickets/{ticket_id} – Delete Ticket

Path parameter: ticket_id (UUID). Success: 200 OK, body {"message": "Ticket deleted successfully"}. Error responses: 404 Not Found ("Ticket not found"); 422 Unprocessable Entity for an invalid UUID.

3.1.2 AI Assistance Module (app/api/ai.py)

Router prefix: /ai. Provides AI-generated ticket summarization and a suggested response, backed by Amazon Bedrock.

Endpoint	Method	Purpose	Success Code
/ai/summarize	POST	Summarize a ticket description and suggest a response using an LLM	200 OK

Request body (SummarizeRequest): ticket_description (string, 10–5000 chars, required).

```
{
  "ticket_description": "User reports the mobile app crashes on login after the latest update."
}
```

Response (200, SummarizeResponse): summary (string), suggested_response (string). The BedrockService builds a prompt from the TICKET_SUMMARY_V1 template, calls client.converse() on the configured Bedrock model (BEDROCK_MODEL_ID), strips any Markdown code fences from the model output, and parses the result as JSON containing "summary" and "suggested_response" keys.

Error responses: 502 Bad Gateway ("The AI service is temporarily unavailable") when the Bedrock call raises BotoCoreError/ClientError, or when the model's response is missing expected keys or is not valid JSON.

3.1.3 System Endpoints (app/main.py)

Endpoint	Method	Purpose	Success Code
/health	GET	Liveness check – returns {"status": "ok"}	200 OK
/ready	GET	Readiness check – executes SELECT 1 against the database	200 OK / 503 Service Unavailable

/ready returns 200 with {"status": "ready", "database": "connected"} when the database connection succeeds, or 503 with {"status": "not_ready", "database": "unavailable"} if the connection attempt raises an exception. Every response also carries an X-Response-Time header (milliseconds) added by middleware in app/main.py.

The application exposes two functional modules: Ticket Management (full CRUD over support tickets) and AI Assistance (LLM-generated ticket summary and suggested response). System endpoints (/health, /ready) support operational monitoring.

3.2 Inputs/Outputs

Schema	Field	Type	Constraints
TicketCreate	title	str	1–200 chars, whitespace-stripped, required
TicketCreate	description	str	1–5000 chars, whitespace-stripped, required
TicketCreate	priority	Priority enum	High Medium Low, default Medium
TicketCreate	assignee_email	str None	optional, no format validation
TicketUpdate	title / description / priority / status	optional	only supplied fields are applied
TicketResponse	id, title, description, priority, assignee_email, status, created_at, updated_at	–	from_attributes=True (built from the ORM object)
SummarizeRequest	ticket_description	str	10–5000 chars, required
SummarizeResponse	summary, suggested_response	str	produced by the Bedrock model

3.3 Business Rules

- New tickets default to priority Medium and status Open if not specified.
- Priority must be exactly one of High, Medium, or Low (case-sensitive); Status must be exactly one of Open, In_progress, or Closed (case-sensitive) – enforced by Pydantic/SQLAlchemy enums.
- Tickets whose status is Closed cannot be updated (enforced in TicketService.update_ticket, raising ClosedTicketError).
- Ticket updates are partial: only the fields explicitly supplied in the request body are changed.
- Title and description values are automatically trimmed of leading/trailing whitespace (str_strip_whitespace) on TicketCreate.
- Ticket summarization requires a ticket_description of at least 10 characters.

3.4 Error Handling

Condition	Exception / Mechanism	HTTP Status	Message
Ticket not found (get/update/delete)	TicketNotFoundError (app/services/ticket_service.py)	404	Ticket not found
Update attempted on a Closed ticket	ClosedTicketError	400	Closed tickets cannot be updated
Invalid request payload / path parameter	Pydantic / FastAPI validation	422	FastAPI's default validation error body
Bedrock call fails or returns an unparsable response	BedrockServiceError (app/services/aws_service/bedrock.py)	502	The AI service is temporarily unavailable

Database unreachable at readiness check	generic Exception caught in /ready	503	{"status": "not_ready", "database": "unavailable"}
---	------------------------------------	-----	--

The `get_db()` dependency (`app/core/deps.py`) rolls back the database session automatically if any exception is raised while handling a request, and commits it otherwise.

4. Non-Functional Requirements

4.1 Performance

- All ticket and AI endpoints are implemented as async request handlers, and database access uses SQLAlchemy's async engine with the `asynpg` driver.
- A middleware in `app/main.py` measures per-request elapsed time and reports it via an `X-Response-Time` response header.
- A Locust load-testing scenario (`locust.py`) exercises `GET /health`, `GET /tickets`, and `POST /tickets` under simulated concurrent users.
- A standalone cProfile-based script (`profile_test.py`) profiles the ticket service's `create/list/get/update/delete` flow.

4.2 Security

- `CORSMiddleware` restricts allowed origins to `http://localhost:3000` (`allow_credentials=True`, all methods/headers allowed).
- AWS credentials (`AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY`) are read from environment configuration and used only server-side to construct the boto3 Bedrock client; they are never exposed in API responses.
- The `Settings` class defines `SECRET_KEY` and `ACCESS_TOKEN_EXPIRE_MINUTES` fields.
- No authentication or authorization mechanism (e.g. API keys, OAuth, JWT, session cookies) protects any of the `/tickets` or `/ai` endpoints.

4.3 Availability

- `GET /health` provides a simple liveness probe returning `{"status": "ok"}`.
- `GET /ready` provides a readiness probe that executes `SELECT 1` against the database and reports 200/503 accordingly.
- The Dockerfile defines a `HEALTHCHECK` that polls `http://127.0.0.1:8000/health` every 30 seconds (3s timeout, 5s start period, 3 retries).

4.4 Maintainability

- The codebase follows a layered architecture (`router` → `service` → `repository` → `ORM model`), keeping HTTP concerns, business rules, and data access separate.

- Database schema changes are versioned through Alembic migrations (alembic/versions/).
- Configuration is centralized in a single pydantic-settings Settings class (app/core/config.py), sourced from environment variables or a .env file.
- An automated pytest suite (unit and integration tests) exercises schema validation and full API request/response behavior.

5. Data Model / Database Design

5.1 Entity-Relationship Diagram

The source code defines a single database entity, tickets (app/models/ticket.py), with no foreign keys or relationships to other tables. A multi-entity ER diagram is therefore not applicable; the single-table structure is documented in section 5.2 below.

5.2 Tables and Fields

Table: tickets

Column	Type	Constraints
id	UUID (PostgreSQL UUID)	Primary key, default generated via uuid4()
title	VARCHAR(200)	NOT NULL
description	VARCHAR (unbounded String)	NOT NULL
priority	Enum(Priority): High, Medium, Low	NOT NULL, default Medium
status	Enum(Status): Open, In_progress, Closed	NOT NULL, default Open
created_at	TIMESTAMP WITH TIME ZONE	NOT NULL, server_default now()
updated_at	TIMESTAMP WITH TIME ZONE	NOT NULL, server_default now(), updated onupdate now()
assignee_email	VARCHAR(254)	Nullable

assignee_email was added after the initial table creation via the Alembic migration df08d1fab4e1 ("add assigne email").

5.3 Indexes and Keys

id is the table's primary key (UUID). No additional indexes, unique constraints, or foreign keys are defined in the source code beyond the primary key.

5.4 Sample Data

```
{
  "id": "3f9a9f0e-2c3b-4a11-9b34-9d2e1a6d7c11",
  "title": "Login failed",
  "description": "Employee login was declined",
  "priority": "High",
  "assignee_email": "support@example.com",
  "status": "Open",
  "created_at": "2026-07-24T09:00:00+00:00",
```

```
"updated_at": "2026-07-24T09:00:00+00:00"
}
```

6. Integration Points

6.1 External Systems

Amazon Bedrock Runtime is the only external system integration are used in the source code. It is used exclusively by BedrockService (app/services/aws_service/bedrock.py) to generate AI ticket summaries and suggested responses. A PostgreSQL database is also an external dependency, but is treated as the application's primary datastore rather than a third-party integration.

6.2 APIs Used

A FakeBedrockService class is also present in the same module, offering a deterministic offline stand-in for the real Bedrock client (useful for demonstrations or tests without live AWS calls), though the AI router currently instantiates the real BedrockService directly.

- Amazon Bedrock Runtime – Converse API (boto3 bedrock-runtime client, client.converse()), invoked with the model id from the BEDROCK_MODEL_ID setting, maxTokens=350 and temperature=0.1.
- The application itself exposes /tickets and /ai/summarize as consumable REST APIs (see section 3.1), plus /health and /ready for monitoring.

6.3 Data Flow Diagram

Ticket flow:

```
Client -> FastAPI(CORS + timing middleware) -> Ticket Router
      -> TicketService -> TicketRepository -> AsyncSession -> PostgreSQL
```

AI summarization flow:

```
Client -> FastAPI -> AI Router -> BedrockService
      -> Amazon Bedrock Runtime (Converse API) -> JSON parse -> SummarizeResponse
```

6.4 Authentication/Authorization

No login mechanism, OAuth/JWT tokens, API keys, or role-based access controls were found protecting any endpoint; all routes in app/api/tickets.py and app/api/ai.py are open/unauthenticated as implemented.

7. Test Strategy

7.1 Test Scenarios

Test File	Type	Coverage
-----------	------	----------

tests/unit/test_ticket_create.py	Unit	TicketCreate validation: title/description length limits, whitespace trimming, valid/invalid/default priority, optional assignee_email
tests/unit/test_ticket_update.py	Unit	TicketUpdate validation: partial updates, valid/invalid priority and status values, case sensitivity, type errors
tests/integration/test_ticket_create_api.py	Integration	POST /tickets/ – success cases, default priority, valid/invalid priority values, missing/null required fields, invalid types, special characters, response shape
tests/integration/test_ticket_get.py	Integration	GET /tickets/{id} – success, not found, invalid UUID, multiple tickets, behavior after delete, response shape
tests/integration/test_ticket_update_api.py	Integration	PUT /tickets/{id} – partial updates, not found, invalid priority, invalid UUID, empty body
tests/integration/test_ticket_delete.py	Integration	DELETE /tickets/{id} – success, not found, double delete, invalid UUID, isolation between tickets, response message

The project includes both unit tests (schema validation, no database) and integration tests (full request/response cycle against a real test database via an ASGI test client).

7.2 Expected Results

- Valid requests to POST /tickets/ return 201 Created with a TicketResponse body matching the submitted data and defaults.
- Invalid payloads (bad title/description length, invalid enum values, missing/null required fields, wrong types) return 422 Unprocessable Entity.
- GET/PUT/DELETE against a non-existent or already-deleted ticket id return 404 Not Found.
- GET/PUT/DELETE with a syntactically invalid UUID path parameter return 422 Unprocessable Entity.
- Deleted tickets are no longer retrievable (subsequent GET returns 404); other tickets remain unaffected by a delete.

7.3 Traceability to Requirements

Requirement / Endpoint	Covered By
TicketCreate validation rules (3.3)	tests/unit/test_ticket_create.py
TicketUpdate validation rules (3.3)	tests/unit/test_ticket_update.py
POST /tickets/ (3.1.1)	tests/integration/test_ticket_create_api.py
GET /tickets/{id} (3.1.1)	tests/integration/test_ticket_get.py
PUT /tickets/{id}, closed-ticket rule (3.1.1, 3.3)	tests/integration/test_ticket_update_api.py
DELETE /tickets/{id} (3.1.1)	tests/integration/test_ticket_delete.py

8. Deployment & Configuration

8.1 Environments

This is a Dev environment. A single Docker image (built from the provided Dockerfile) is configured entirely through environment variables read by the Settings class. The test suite uses a separate database, derived by replacing "ai_service_desk" with "ai_service_desk_test" in DATABASE_URL (tests/conftest.py) – effectively a dedicated test environment for the database only.

8.2 Setup Steps

1. Install dependencies: `pip install -r requirements.txt` (or use the `uv.lock/pyproject.toml` with `uv`).
2. Provide the required environment variables (see 8.3) via a `.env` file or the shell environment.
3. Apply database migrations: `alembic upgrade head`.
4. Run the application locally: `uvicorn app.main:app --reload` (or, as in the Dockerfile, `uvicorn app.main:app --host 0.0.0.0 --port 8000`).
5. Run the automated test suite: `pytest`.
6. (Optional) Run the Locust load test: `locust -f locust.py`.
7. (Optional) Build and run the Docker image: `docker build -t ai-service-desk . && docker run -p 8000:8000 --env-file .env ai-service-desk`.

8.3 Configuration Files and Parameters

Settings are declared in `app/core/config.py` using `pydantic-settings`, loaded from environment variables or a `.env` file (case-insensitive; unknown keys are ignored).

Variable	Type	Default	Purpose
APP_NAME	str	"AI SERVICE DESK"	Application name
API_VERSION	str	"1.0.0"	API version string
DEBUG	bool	required, no default	Debug flag
DATABASE_URL	str	required, no default	SQLAlchemy async database connection string
SECRET_KEY	str	required, no default	Declared in Settings
ACCESS_TOKEN_EXPIRE_MINUTES	int	60	Declared in Settings
AWS_ACCESS_KEY_ID	str	required, no default	AWS credential for the Bedrock client
AWS_SECRET_ACCESS_KEY	str	required, no default	AWS credential for the Bedrock client
AWS_REGION	str	required, no default	AWS region for the Bedrock client
BEDROCK_MODEL_ID	str	required, no default	Bedrock model id used in BedrockService
AWS_DEMO_MODE	str	required, no default	Declared in Settings; usage not found in the reviewed source code

DATABASE_READY	str	required, no default	Declared in Settings; usage not found in the reviewed source code
----------------	-----	----------------------	---

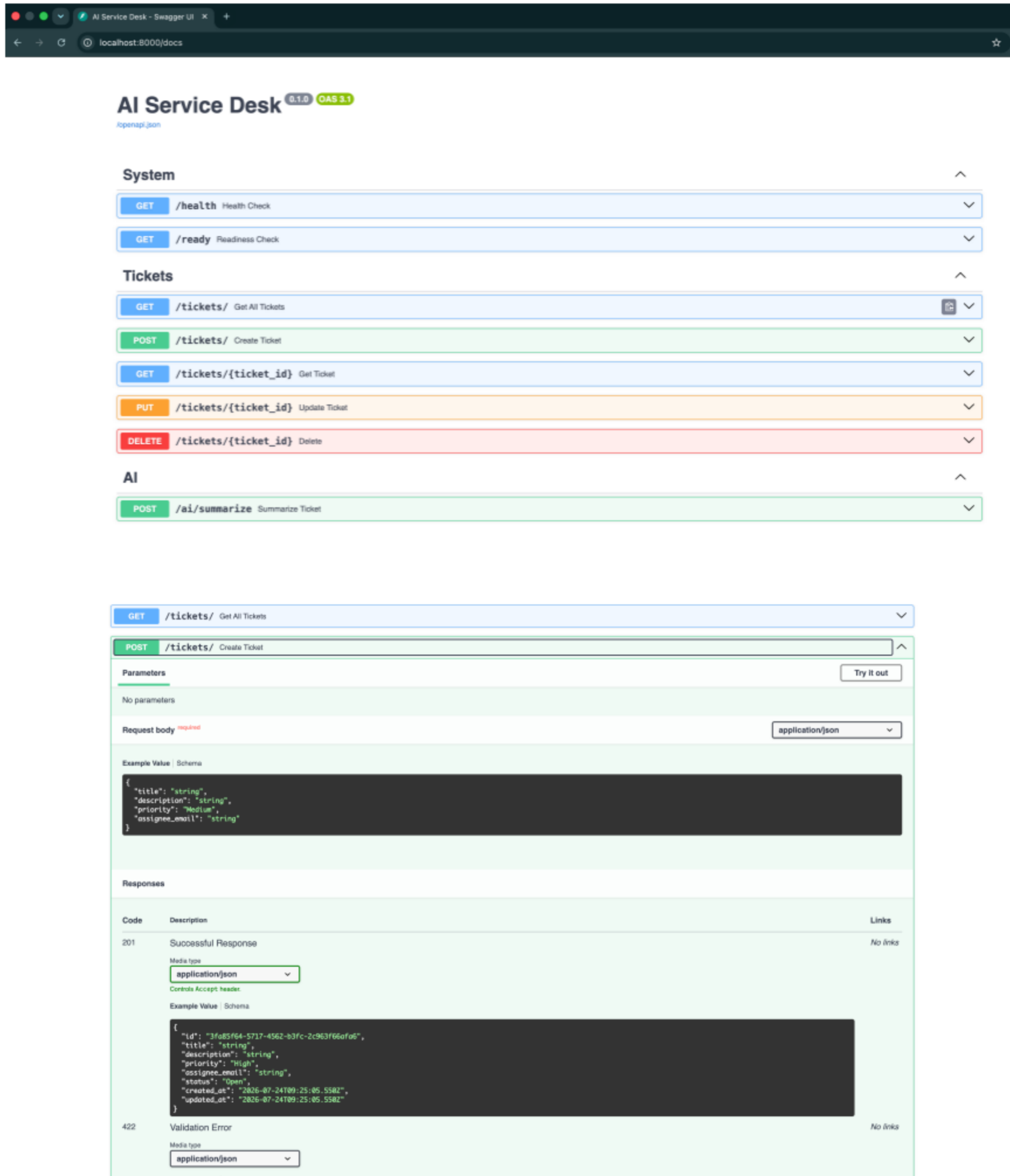
Other relevant configuration/deployment files: Dockerfile (container build and runtime), alembic.ini (migration script location and logging configuration), pyproject.toml / requirements.txt / uv.lock (Python dependencies).

9. Appendix

Glossary

Term	Definition
Ticket	A support request record stored in the tickets table, with a title, description, priority, status, and optional assignee_email
Priority	Enum field on a ticket: High, Medium, or Low
Status	Enum field on a ticket: Open, In_progress, or Closed
Repository (pattern)	A class (TicketRepository) that encapsulates direct database queries for an entity
Service (layer)	A class (TicketService, BedrockService) that implements business logic, sitting between the router and repository/external API
Async Session	A SQLAlchemy AsyncSession representing one request-scoped, awaitable database transaction
Bedrock	Amazon's managed service for accessing foundation LLMs, used here via the Converse API
Alembic Migration	A versioned script describing an incremental change to the database schema

Screenshots



The screenshot displays the Swagger UI for the AI Service Desk API. The browser address bar shows the URL `localhost:8000/docs`. The API title is "AI Service Desk" with versions "0.1.0" and "OAS 3.1" indicated. The API is defined in `openapi.json`.

The API endpoints are categorized into three groups:

- System**
 - `GET /health` Health Check
 - `GET /ready` Readiness Check
- Tickets**
 - `GET /tickets/` Get All Tickets
 - `POST /tickets/` Create Ticket
 - `GET /tickets/{ticket_id}` Get Ticket
 - `PUT /tickets/{ticket_id}` Update Ticket
 - `DELETE /tickets/{ticket_id}` Delete
- AI**
 - `POST /ai/summarize` Summarize Ticket

The details for the `POST /tickets/` endpoint are expanded, showing the following information:

- Parameters:** No parameters.
- Request body:** Required, `application/json`.
- Example Value / Schema:**

```
{
  "title": "string",
  "description": "string",
  "priority": "Medium",
  "assignee_email": "string"
}
```
- Responses:**

Code	Description	Links
201	Successful Response	No links
422	Validation Error	No links

Code Snippets

Ticket ORM model (app/models/ticket.py):

```
class Ticket(Base):
    __tablename__ = "tickets"
    id: Mapped[UUID] = mapped_column(PGUUID(as_uuid=True), primary_key=True, default=uuid4)
    title: Mapped[str] = mapped_column(String(200), nullable=False)
    description: Mapped[str] = mapped_column(String, nullable=False)
    priority: Mapped[Priority] = mapped_column(SQLEnum(Priority), default=Priority.MEDIUM, nullable=False)
    status: Mapped[Status] = mapped_column(SQLEnum(Status), default=Status.OPEN, nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), server_default=func.now(),
    nullable=False)
    updated_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), server_default=func.now(),
    onupdate=func.now(), nullable=False)
    assignee_email: Mapped[str | None] = mapped_column(String(254), nullable=True)
```

Closed-ticket business rule (app/services/ticket_service.py):

```
async def update_ticket(self, ticket_id: UUID, ticket: TicketUpdate):
    exist_ticket = await self.repository.get(ticket_id)
    if exist_ticket is None:
        raise TicketNotFoundError("Ticket not found")
    if exist_ticket.status == Status.CLOSED:
        raise ClosedTicketError("Closed tickets cannot be updated")
    update_data = ticket.model_dump(exclude_unset=True)
    for field, value in update_data.items():
        setattr(exist_ticket, field, value)
    return await self.repository.update(exist_ticket)
```

References

- FastAPI documentation – <https://fastapi.tiangolo.com/>
- SQLAlchemy 2.0 documentation – <https://docs.sqlalchemy.org/>
- Alembic documentation – <https://alembic.sqlalchemy.org/>
- AWS Bedrock Runtime documentation – <https://docs.aws.amazon.com/bedrock/>